PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00
Log In Create An Account

This forum is sponsored by:
Moonbase.sh
Everything you need to sell & deliver plugins
Licensing
Payments
E-Commerce
Marketing tools

What's New @ KVR Audio?

Fastest way to interpolate phase & magnitude between two complex spectra

Post Reply New Topic

13 posts • Page **1** of **1**

RELATED PRODUCTS Products related to this topic can

TheWaveWarden
KVRist

Topic Starter

306 posts since 10 Apr, 2020

Mon Jan 22, 2024 11:05 am 0

I'm interpolating between two fourier bin arrays which represent wavetables in my synth. Doing linear interpolation

KVR PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

complex numbers is far apart, making them in the worst case cross zero (for example interpolating from {1, 0} to {-1, 0}). A bin of complex zero of course results in no sinusoid output for this bin, which is undesirable.

I've come to the realization that retrieving the angle and amplitude, then interpolating these two and reconstructing a new complex number would circumvent this. However, my wavetables are 1024 bins long, and iFFT happens rather often if parameters are being modulated. The "trivial" solution would go like this:

CODE: SELECT ALL

```
get phases (2x arctan)
get magnitudes (2x sqrt)
interpolate
reconstruct complex number (sir
```

so in summary that's six expensive functions per bin.

I'm trying to wrap my head around if there's a faster way to achieve the above.

I've thought about faking the result above, to make it at least better than linear interpolation. For example one could just do linear interpolation and then

KVR PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

outward" in the process.
This could also further be improved by approximating the magnitude (L2 norm) with L1 or L_infinity norms.

However, these introduce rather big discrepancies to the original formula. I've made a super messy desmos graph showing the different approaches in action in case anybody's interested:

https://www.desmos.com/calculator/ftcxe6drqu
Purple: Linear interpolation
Blue: Phase & angle interpolation
Green: "Push out" L2
Red: "Push out" L1
Orange: "Push out" L_infty

Happy to hear if there's any other methods or resources on what I'm after 🙂

Last edited by TheWaveWarden on Tue Jan 23, 2024 8:03 am, edited 2 times in total.

Odin 2: Semi Modular Synthesizer
Filter Force: Dual Creative Filter Bank
Spline Wavemorphing Synthesizer

Music Engineer
KVRAF

4342 posts since 8 Mar, 2004 from Berlin, Germany

KVR PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

> **TheWaveWarden wrote:** ↑ Mon Jan 22, 2024 11:05 am
>
> CODE: SELECT ALL
>
> ```
> get ph
> get ma
> interp
> recons
> ```
>
> so in summary that's six expensive functions per bin.
>
> I'm trying to wrap my head around if there's a faster way to achieve the above.

It would seem to me that magnitude and phase can be precomputed when the waveforms are loaded/generated, so that's not really something that you need to do for each update interval. The sin/cos calls could be replaced by trigonometric recurrences - or perhaps, more elegantly, a single, complex-valued recursion of the form z[n] = z[n-1] * z[1] where z[0] = 1 + i*0, z[1] = exp(i*w) = cos(w) + i*sin(w) and w = 2*pi/N (if I remember correctly). I may be missing something, though. Just a shoot from the hips. Such trig recursions are useful whenever you want to evaluate multiple sines and cosines and the angles are related by being all multiples

different use cases, too - but that's the most common one and the one that we happen to have here). There is a paper somewhere that investigates other trig recurrence formulas that may be better behaved numerically (i.e. have less roundoff error accumulation) for slightly higher computational cost. I currently don't remember the name of that paper, so maybe someone else could post a link?

Edit - here are some resources that seem to be relevant for this recursive sine oscillator business:

https://ccrma.stanford.edu/~dattorro/Ef ... nPart3.pdf
https://www.njohnson.co.uk/pdf/drdes/Chap7.pdf
https://ccrma.stanford.edu/~jos/pasp/Di ... ators.html

but I'm not sure, if the paper that I initially thought of is one of them.

...well, actually, it looks like you could also precompute all the sines and cosines and store them in a table, too. You wouldn't even need to recompute them when the table changes (only when the table

KVR PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

1024 bins that would be a table size of 2048...aha - yeah - that's what I use, too)

My website: rs-met.com, My presences on: YouTube, GitHub, Facebook

mystran
KVRAF

8307 posts since 12 Feb, 2006 from Helsinki, Finland

Mon Jan 22, 2024 10:44 pm 0

> **Music Engineer wrote:** ↑ Mon Jan 22, 2024 3:40 pm
>
> > **TheWaveWarden wrote:** ↑ Mon Jan 22, 2024 11:05 am
> >
> > CODE: SELECT ALL
> >
> > ```
> > get phases (2x arctan)
> > get magnitudes (2x sqr
> > interpolate
> > reconstruct complex nu
> > ```
> >
> > so in summary that's six expensive functions per bin.
> >
> > I'm trying to wrap my head around if there's a faster way to achieve the above.

KVR PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

> precomputed when the waveforms are loaded/generated, so that's not really something that you need to do for each update interval.

There is a subtle issue though: because phase wraps around, you can always interpolate in two directions and in the worst-case (which happens to be the same case where linear interpolation would pass through zero) both paths have the same length.

Interpolating linear magnitude is also less than ideal, because magnitude is not linear in perception... so really you'd probably want to interpolate the complex logarithm (with phase found in imaginary component).. but that's expensive then (well, approximated exp isn't too bad, but still) and now we need to bias slightly away from zero to keep the logarithms finite.. and that doesn't solve the two-path issue, or the case where we have a large value with phase 0 and then a tiny value near noise floor that happens to have a phase of (almost exactly) pi.. now the target value is perceptually zero, yet because of rounding it causes a large change in phase.

in graphics with quaternions: rather than bothering with proper spherical interpolation, do a lerp and normalize (and in this case with varying amplitude, scale with interpolated magnitude). That's just an extra sqrt, those are not that expensive. With quaternions there's a little gotcha with double-cover, but with complex numbers that's not relevant and

through zero only if phase changes exactly 180 degrees).

I'd probably just interpolate the complex values in cartesian coordinates though, but perhaps either compute more intermediate shapes or perform cubic (eg. hermite) interpolation. It turns out that if we sample continuously changing phase at a reasonable rate, then cubic interpolation will trace a pretty reasonable path even where linear would take a less-than-ideal shortcut (ie. you need a LOT less intermediate steps compared to linear). That's twice as much data to fetch though (and a bunch of multiply-adds)... and I guess it won't work if tables are computed on the fly.

https://github.com/signaldust

KVR PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

TheWaveWarden
KVRist

Topic Starter

306 posts since 10 Apr, 2020

Tue Jan 23, 2024 8:02 am 0

Thanks for joining the discussion! 😀

> **Music Engineer wrote:** ↑ Mon Jan 22, 2024 3:40 pm
> The sin/cos calls could be replaced by trigonometric recurrences - or perhaps, more elegantly, a single, complex-valued recursion of the form z[n] = z[n-1] * z[1] where z[0] = 1 + i*0, z[1] = exp(i*w) = cos(w) + i*sin(w) and w = 2*pi/N (if I remember correctly).

This would be useful if I did consecutive updates at even steps on the interpolation. Then I could reuse the same "complex rotational operator" to calculate the next step. However, my synth does for example demand a wavetable at position 10.7, this means 30% of bin [10] and 70% of bin [11], and the next call might be at a completely different, arbitrary position between different bins. Sorry

> **Music Engineer wrote:** ↑ Mon Jan 22, 2024 3:40 pm
> It would seem to me that magnitude and phase can be precomputed when the waveforms are loaded/generated, so that's not really something that you need to do for each update interval

Ah very nice, I could just convert all the cartesian bins to polar, right. Would need to keep the option for both thought, since I'm offering linear interpolation as well.

> **mystran wrote:** ↑ Mon Jan 22, 2024 10:44 pm
> do a lerp and normalize (and in this case with varying amplitude, scale with interpolated magnitude). That's just an extra sqrt, those are not that expensive.

This is what I meant in my original post with the "push out" tactic (see purple graph in the desmos demo) 😉

> **mystran wrote:** ↑ Mon Jan 22, 2024 10:44 pm
> I'd probably just interpolate the complex values in cartesian

> intermediate shapes or perform cubic (eg. hermite) interpolation. It turns out that if we sample continuously changing phase at a reasonable rate, then cubic interpolation will trace a pretty reasonable path even where linear would take a less-than-ideal shortcut (ie. you need a LOT less intermediate steps compared to linear). That's twice as much data to fetch though (and a bunch of multiply-adds)... and I guess it won't work if tables are computed on the fly.

I thought about this as well, but as I said above, I only need to get the interpolated value at some arbitrary point one single time. Now lets say I construct one extra support point intbetween: Finding this point is kinda the same problem as finding the interpolated value, no?

Odin 2: Semi Modular Synthesizer
Filter Force: Dual Creative Filter Bank
Spline Wavemorphing Synthesizer

S0lo
KVRian

> **TheWaveWarden wrote:** ↑ Mon Jan 22, 2024 11:05 am
> unless the angle/phase of the complex numbers is far apart, making them in the worst case cross zero (for example interpolating from {1, 0} to {-1, 0}). A bin of complex zero of course results in no sinusoid output for this bin, which is undesirable.

Just curious, If passing through zero (no sinosoid) is only for some bins (obviously not for the whole spectrum) then why would that be undesirable. If you fight it, wouldn't you be missing out on those sounds that go silent **per bin** then return to some gain? I mean, some one could want that. (Edit: such as rythmic or percussive sounds).

My post is obviously not an answer to your question so I apologize in advance.

---

www.solostuff.net
Advice is heavy. So don't send it like a mountain.

mystran
KVRAF

KVR PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

Tue Jan 23, 2024 1:40 pm 0

> **TheWaveWarden wrote:** ↑ Tue Jan 23, 2024 8:02 am
> I thought about this as well, but as I said above, I only need to get the interpolated value at some arbitrary point one single time. Now lets say I construct one extra support point intbetween: Finding this point is kinda the same problem as finding the interpolated value, no? 😀

That depends. If it can be precomputed, then the cost of computing such points no longer really matters. For morphing wavetables produces algorithmically it might also be possible to just produce a higher resolution table directly.

In any case, there's no reason to take two arcus tangents ever. If we treat the complex numbers as 2D vectors and normalize them, then dot-product gives cosine and cross-product gives sine of the *angle between them*. So even if you were to take atan2 of this pair, now you only need one *and* we avoid having to worry about which way we're going, 'cos we'll get angle between [-pi,pi] which is the

and the way you normalize might give you more options in terms of what happens when one of the values is very closer to zero.. though converting to angles is always a bit ugly in terms of both numerical and computational performance, so .. idk, I'd perhaps avoid it.

https://github.com/signaldust

signalsmith
KVRist

121 posts since 5 Jul, 2018 from Cambridge, UK

Tue Jan 23, 2024 4:16 pm 0

If you were interpolating between bins within a spectrum, you'd want to make sure that your "reference time" (t=0, which the phases of each bin are relative to) matched where the input's energy was going to be. For example you might rotate your FFT input around in time so that the window is centred on t=0, instead of t=N/2.

(As a motivating example: consider the case where our input is an impulse, at the centre of your window. All your bins will have magnitude 1, and you'd expect all of your interpolated values to as well, right? But if

KVR PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

alternate sign, i.e. they'll have completely opposite phase, so simple interpolation won't work well.

I think a neat way to phrase this is thinking of "time-response" of each bin-interpolation method, which is the exact counterpart to the "frequency response" of that method when doing fractional delays.)

There's a similar concern when interpolating a particular bin across time (e.g. for fractional delays within a spectral delay) - you want to make sure all the spectra have a common reference time (t=0) for their phases.

However, if you're interpolating between two unrelated wavetables, there isn't really an inherent common reference time, so I'm not sure there's a better approach than interpolating the complex values and then correcting the energy.

Signalsmith Audio | We have a blog! | Check out Utopia, our spectral reverb

JustinJ
KVRist

KVR PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

Tue Jan 23, 2024 9:44 pm 0

FWIW, spectral morphing between FFTs is an option in Serum's wavetable editor. It's in U-he Hive2 as a live wavetable interpolation option and it's also in the Vital source code. I pretty much implemented it as per the way it's done in Vital. Meaning, there's atan2 and sqrt to optimise but that's not *too* hard. It's still a CPU hit compared to time domain interpolation but it does produce an audibly smoother interpolation between waves.

TheWaveWarden
KVRist

Topic Starter

306 posts since 10 Apr, 2020

Thu Jan 25, 2024 9:32 am 0

> **mystran wrote:** ↑ Tue Jan 23, 2024 1:40 pm
> In any case, there's no reason to take two arcus tangents ever. If we treat the complex numbers as 2D vectors and normalize them, then dot-product gives cosine and cross-product gives sine of the *angle between them*. So even

> one *and* we avoid having to worry about which way we're going, 'cos we'll get angle between [-pi,pi] which is the shortest rotation of the first vector to get the second vector...

Awesome, this is exactly what I was looking for! Also didn't realize there is a 2D cross product. With scalar output even...

> **signalsmith wrote:** ↑ Tue Jan 23, 2024 4:16 pm
> If you were interpolating between bins within a spectrum, you'd want to make sure that your "reference time" (t=0, which the phases of each bin are relative to) matched where the input's energy was going to be. For example you might rotate your FFT input around in time so that the window is centred on t=0, instead of t=N/2.

First of all many thanks for your ADC talks and the guides on your homepage. It's been an awesome resource to me. Absolutely love the visual representation in the reverb blog. Basically got me from several failed attempts at

KVR PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

My first thought was to just phase align all wavetables to have the same phase to begin with - problem solved. However as it turns out for example the standard analog triangle wave itself is not phase aligned. If you do so, it sounds exactly identical of course, but looks really weird to the user (there's a wave display in my synth). And my guess is if the user selects triangle they wanna have just that.. Plus the user can import or draw custom wavetables, so phase aligning isn't really an option anymore for me.

Here's the classic and phase-aligned triangles:
triangle_phase.png

> **S0lo wrote:** ↑
> Just Tue Jan 23, 2024 11:50 am
> curious, If passing through zero (no sinosoid) is only for some bins (obviously not for the whole spectrum) then why would that be undesirable. If you fight it, wouldn't you be missing out on those sounds that go silent per bin then return to some gain? I mean, some one could want that. (Edit: such as rythmic or percussive sounds).

bin will just vanish amidst the interpolation and the reappear for no apparent sonic reason. I guess it's more intuitive for the ear to have the amplitude increase steadily from start to finish.

For example as stated above the bins of the triangle wave don't all have the same phase, so lerping from square to triangle would have bins [3, 7, 11, ...] all vanish and then reappear, which looks and sounds weird. If the user wants it they can choose the time interpolation option in my case at least.

You do not have the required permissions to view the files attached to this post.

Odin 2: Semi Modular Synthesizer
Filter Force: Dual Creative Filter Bank
Spline Wavemorphing Synthesizer

S0lo
KVRian

1309 posts since
31 Dec, 2008

Thu Jan 25, 2024 11:16 am 0

Post removed. Never mind. Brain fa*t

Last edited by S0lo on Thu Jan 25, 2024 1:16 pm, edited 1 time in total.

mountain.

Z1202
KVRAF

1607 posts since
12 Apr, 2002

Thu Jan 25, 2024 12:18 pm 0

> **TheWaveWarden wrote:** ↑ Thu Jan 25, 2024 9:32 am
> My first thought was to just phase align all wavetables to have the same phase to begin with - problem solved. However as it turns out for example the standard analog triangle wave itself is not phase aligned. If you do so, it sounds exactly identical of course

Generally speaking, changing the phase content does change the sound (just not always, I'm not sure what are the conditions). Further, some waveforms (in particular, those with discontinuities) can become infinitely high (if not bandlimited, or very high if bandlimited) with some phase settings.

mystran
KVRAF

KVR PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

Thu Jan 25, 2024 1:52 pm 0

> **TheWaveWarden wrote:** ↑
> Thu Jan 25, 2024 9:32 am
>
> > **mystran wrote:** ↑
> > Tue Jan 23, 2024 1:40 pm
> > In any case, there's no reason to take two arcus tangents ever. If we treat the complex numbers as 2D vectors and normalize them, then dot-product gives cosine and cross-product gives sine of the *angle between them*. So even if you were to take atan2 of this pair, now you only need one *and* we avoid having to worry about which way we're going, 'cos we'll get angle between [-pi,pi] which is the shortest rotation of the first vector to get the second vector...
>
> Awesome, this is exactly what I was looking for! Also didn't realize there is a 2D cross product. With scalar output even...

In a sense there really isn't a separate 2D cross-product as

KVR PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

(3D) cross-product being orthogonal necessarily points along the z-axis (the rotation axis). We need a third dimension to represent this *vector*, but we can compute it's *signed magnitude* (which is really just the z component, because x and y components are zero due to orthogonality) which happens to be the product of the magnitudes of the two vectors times the sine of the angle between them. Restricting ourselves onto the xy-plane though, we can only ever rotate around the z-axis anyway, so there's nothing lost by treating this (scaled, if the vectors aren't normalized) "sine" as a scalar... so that's your 2D cross-product. 🙂

edit: I'd like to observe that the dot- and cross-products together are also sufficient to fully represent rotations in 3D (and in fact you can trivially build quaternions out of them; multiply the vector by ijk and you'll have a quaternion to rotate for twice the angle; we won't go into the details of why twice the angle, but with unit quaternions getting half-angle is just a matter of averaging with identity and renormalizing, so not a big deal), but since cross-

xy-components are zero if our rotation restricted to the xy-plane (the rotation axis must be z), so all we really care in the 2D case is the sign and the magnitude.

https://github.com/signaldust

Urs
u-he

29425 posts since 8 Aug, 2002 from Berlin

Mon Jan 29, 2024 4:59 pm 0

> **JustinJ wrote:** ↑
> FWIW, Tue Jan 23, 2024 9:44 pm spectral morphing between FFTs is an option in Serum's wavetable editor. It's in U-he Hive2 as a live wavetable interpolation option and it's also in the Vital source code. I pretty much implemented it as per the way it's done in Vital. Meaning, there's atan2 and sqrt to optimise but that's not *too* hard. It's still a CPU hit compared to time domain interpolation but it does produce an audibly smoother interpolation between waves.

quite similar. Indeed for each bin linearly interpolate phase (shortest direction) and magnitude sounds pretty good.

We keep the wavetable as phase/mag per bin in memory when we do this thing. Then we only need a reasonably fast sincos approximation (Pade or something) in SIMD (SSE/NEON) which contains 1 divide for every 4 bins.

Furthermore, at the time we do this we know how many overtones there'll be in the band limited audio, so we commonly have a lot fewer than 1000 bins. More like 100 for chords and stuff. In Hive we do this every 64 samples I think, and then crossfade between two wavetables.

(we also dynamically adjust the wavetable size between 64 samples and 2048 samples in powers of two, depending on how much we need to bandlimit)

u-he.com

Post Reply

13 posts • Page **1** of **1**

Return to "DSP and Plugin Development"

Jump to

PRODUCTS FORUM LP MARKETPLACE DEALS WHAT'S NEW 0 ITEMS | $0.00

Powered by phpBB® Forum Software © phpBB Limited

Privacy | Terms

Submit: News, Plugins, Hosts & Apps | Advertise @ KVR | Developer Account | About KVR / Contact Us | Privacy Statement

© KVR Audio, Inc. 2000-2026